Fonaments d'Informàtica — Resum de teoria

Teoria, esquemes, fórmules i exemples resolts · Temes 1 a 4 · UAB · Curs 2026/27

teoria = targeta blanca | fórmula = caixa blava | «en paraules senzilles» = caixa verda | pas a pas = passos numerats | exemple resolt = caixa groga

Tema 1 · Introducció
1.1 Informàtica i computador
1.2 Unitats d'informació
1.3 Tipus de computadors i Moore
1.4 Estructura del computador
1.5 Llenguatges
1.6 Algorísmica
1.7 Història

Tema 2 · Representació
2.1 Informació · 2.2 Octal
2.3 Bases · 2.4 Canvi de base
2.5 Operar sense signe
2.6 Text · 2.7 Imatge · 2.8 So
2.9 Naturals i enters
2.10 Reals · 2.11 Operar SM/C2
2.12 Carry i overflow

Tema 3 · Components digitals
3.1 Àlgebra de Boole
3.2 Funcions lògiques
3.3 Portes · 3.4 Canòniques
3.5 Karnaugh
3.6 Combinacionals
3.7 Sumadors · 3.8 Mòduls MSI
3.9 Seqüencials

Tema 4 · Estructura
4.1 ISA · 4.2 Von Neumann
4.3 Coll d'ampolla i Harvard
4.4 La CPU · 4.5 Cicle
4.6 Rendiment · 4.7 Memòria
4.8 Busos
4.9 ISA: CISC i RISC

Contingut de l'assignatura: T1 Introducció · T2 Representació · T3 Components digitals · T4 Estructura i arquitectures · T5 Assemblador RISC-V · T6 Microarquitectura · T7a Memòria · T7b E/S · T8 Sistema operatiu. Base de Programació, Sistemes digitals, Arquitectura del computador, Sistemes encastats i Microelectrònica.

TEMA 1 · Introducció

1.1 Informàtica i computador

Informàtica = **Informa**ció + Auto**màtica** (*Computer Science / Engineering*): tractament automàtic de la informació mitjançant ordinadors; inclou disseny de maquinari i programari, gestió de dades i xarxes i automatització de processos.
Computador / ordinador: màquina electrònica digital que fa el processament, l'obtenció, l'emmagatzematge, la transformació i la comunicació de la informació mitjançant **programes preestablerts** (CPU, memòria i dispositius d'E/S).
Dada: conjunt de símbols que expressen un valor numèric, un objecte, un fet o una idea, en la forma adequada per ser processada.

1.2 Unitats d'informació

EN PARAULES SENZILLES

1 bit és un interruptor (0/1). Amb 8 interruptors (1 byte) pots escriure una lletra. Els prefixos SI (kB, MB…) compten de 1000 en 1000; els binaris (KiB, MiB…) de 1024 en 1024.

SI (×1000)	IEC (×1024)	Ordre de mida
kB = 10^3 B	KiB = 2^{10} B	fitxer de text petit
MB = 10^6 B	MiB = 2^{20} B	una foto
GB = 10^9 B	GiB = 2^{30} B	una pel·lícula
TB = 10^{12} B	TiB = 2^{40} B	1000 pel·lícules
PB = 10^{15} B	PiB = 2^{50} B	Google indexat (~100 PB, 2025)
EB = 10^{18} B	EiB = 2^{60} B	trànsit d'internet d'un dia
ZB = 10^{21} B	ZiB = 2^{70} B	tota la informació d'internet
YB = 10^{24} B	YiB = 2^{80} B	

∑ FÓRMULA

$1\ \text{kB} = 10^3\ \text{B} \cdot 1\ \text{KiB} = 2^{10} = 1024\ \text{B}$

n bits → 2^n combinacions

Bit (0/1) → Byte (8 bits) → Paraula (16/32/64 bits: info manipulada en paral·lel)

PAS A PAS

De GB (SI) als GiB que mostra el sistema operatiu

1. Escriu la capacitat en bytes: n GB = $n \cdot 10^9$ B.
2. Divideix entre 2^{30} (1 GiB).
3. El resultat són GiB (sempre menys que els GB anunciats).

EXEMPLE · disc anunciat de 500 GB

$500 \cdot 10^9 / 2^{30} = 500\,000\,000\,000 / 1\,073\,741\,824 \approx$ **465,66 GiB**. (Anàlogament 1 TB ≈ 931,3 GiB.)

1.3 Tipus de computadors i llei de Moore

Personals: propòsit general, cost vs rendiment (portàtil).

Servidors: més capacitat, rendiment i fiabilitat; treball a la xarxa (Google Cloud).

Superordinadors: càlcul científic; clúster de milers de nodes (MareNostrum).

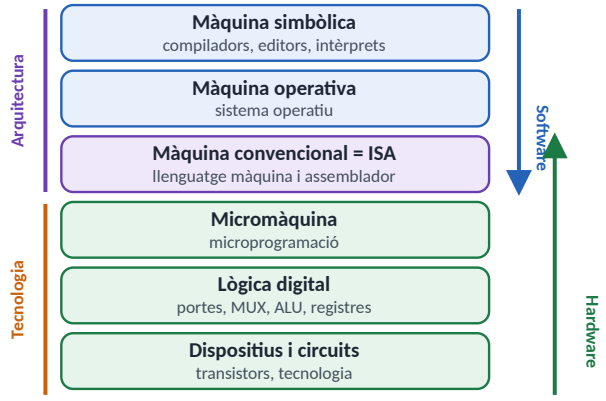
Encastats: petits, aplicacions específiques i temps real; restriccions de rendiment/potència/cost (Raspberry Pi, Jetson).

Mòbils: sense fils, amb bateria, consum restringit (telèfons, tauletes, rellotges).

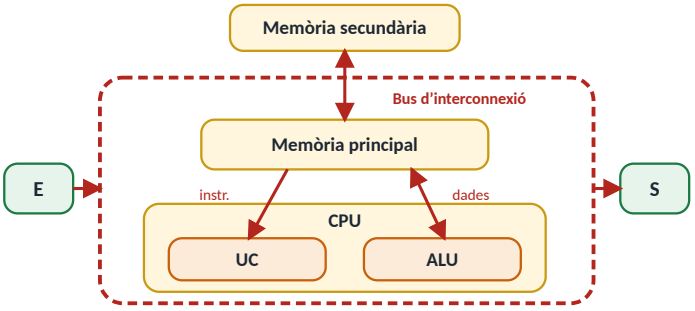
Núvol / boira: recursos compartits (programari i informació) com a servei; l'usuari no controla la infraestructura (Google, Amazon).

Llei de Moore (1965): els recursos de càlcul d'un circuit integrat es dupliquen cada **18-24 mesos** («quants transistors caben en aquest espai de silici»). Cal anticipar-la des del disseny. Ha portat a: **abstracció** · prioritzar el **cas comú** · **paral·lelisme/pipelining** · **algorismes de predicció** (cau) · **jerarquia de memòria** (recursos segons ús/cost) · **redundància** per minimitzar fallades.
Més enllà de Moore: Power Wall (fi de l'escalat de Dennard ~2006: no es pot pujar la freqüència sense sobreescalfar el xip) → **era multinucli** (rendiment limitat per la llei d'Amdahl) → **especialització**: GPU/NPU (llei de Huang), eficiència energètica (llei de Koomey) → **noves tecnologies**: empaquetament 3D i *chiplets* (blocs modulars interconnectats).

1.4 Estructura del computador



Capes: on acaba/comença el HW i el SW?

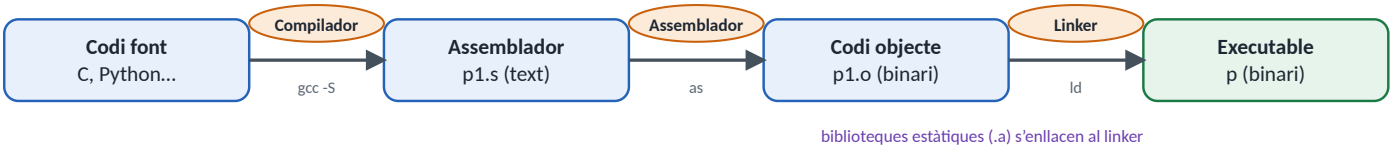


Estructura: dades/instruccions pel bus; la UC controla

Capes: maquinari (processador, memòria, E/S; executa el programari) → programari del sistema (compilador; sistema operatiu: gestiona E/S, memòria, tasques) → programari d'aplicació (alt nivell). **CPU** = UP (unitat de procés / ALU) + UC (unitat de control). **Memòria** (dades i instruccions): registres → cau → memòria principal (RAM) → secundària. **E/S** via bus d'interconnexió.
Placa base: interconnexió física i elèctrica entre CPU, memòria i perifèrics; el **xipset** gestiona la comunicació entre busos; modular (PCIe, SATA, USB). **Encastat: SoC** (CPU+GPU+memòria+controladors d'E/S en un sol xip), filosofia **RISC** (ARM: eficiència i cost), pins **GPIO** per connectar sensors i actuadors.

1.5 Llenguatges de programació

Alt nivell (C/C++, Python, Fortran, Lisp): semblants a la parla habitual, **portables** i independents de la màquina, fàcils de **depurar**. **Compilador:** tradueix el programa font a codi objecte (fitxer); s'enllaça amb biblioteques i es genera el codi màquina; requereix fitxers complets. **Traductors/intèrprets** (Basic, Python): executen el codi font instrucció a instrucció, sense codi objecte.
Baix nivell (assemblador): mnemotècnics del llenguatge màquina; depèn del processador. **Codi màquina:** cadenes de 0 i 1 interpretades pel circuit; cada arquitectura té el seu conjunt d'instruccions; eficient però poc fiable i difícil de transportar.
Sistema operatiu (UNIX, Windows, Linux, Mac OS): programa format per mòduls/subrutines que gestiona les tasques optimitzant el rendiment i controla l'E/S.



Procés de compilació amb gcc: `gcc -O1 p1.c p2.c -o p` *(.c → .s → .o → executable)*

EXEMPLE · mateixa instrucció a dos nivells

Assemblador: `ADD x1, x2, x3` → codi màquina RISC-V: `0000000 00011 00010 000 00001 0110011`. El mateix patró de bits només té sentit per a aquell processador.

1.6 Algorísmica

Algorisme: seqüència de passos per aconseguir quelcom (recepta, manual, ruta). Ha de ser **precís** (no ambigu), **finit**, **determinista** (mateixes dades → mateix resultat) i **coherent** (resol el problema). **Programa:** algorisme escrit per a l'ordinador (instruccions ordenades). **Llenguatge de programació:** normativa i simbologia per redactar programes. **Instrucció:** ordre per al processador: de **transferència** (`ld`, load), de **procés** (`add`) o de **control de flux** (`if`).



Problema → model → algorisme (pseudocodi) → programa → solució, amb verificació



Símbols del diagrama de flux (inici/fi, procés, subrutina, E/S de dades, decisió, referència) units per fletxes que indiquen la seqüència

PAS A PAS

Algorisme d'Euclides (mcd de a i b)

1. Mentre **b ≠ 0**, repeteix els passos 2 i 3.
2. **r ← residu de a entre b.**
3. **a ← b** i **b ← r.**
4. El resultat és **a** (el seu últim valor).

EXEMPLE · mcd(48, 18)

r = 48 mod 18 = 12 → a = 18, b = 12
r = 18 mod 12 = 6 → a = 12, b = 6
r = 12 mod 6 = 0 → a = 6, b = 0 → **mcd = 6.**
Algorisme «parell o senar»: llegir nombre; dividir entre 2; mirar el residu (0 → parell).

1.7 La computació: breu introducció històrica

Àbac (2000-3000 a.C.) i clepsidra → **Euclides** (300 a.C.): algorismes de càlcul → **al-Khwarizmi** (s. IX): popularitza els números àrabs i descriu mètodes per sumar, restar, multiplicar i dividir → **Pascalina** (Pascal, s. XVII): primera calculadora → **Babbage** (s. XVIII/XIX): màquina de diferències (logaritmes) i **màquina analítica**, la primera pensada amb E/S i programa de control (targetes perforades de Jacquard); **Ada Byron** escriu les primeres instruccions (inici dels llenguatges de programació).

Any	Evolució electrònica	Evolució de computadors
1904 / 1943	Vàlvula de buit · 1a generació	Pre-ENIAC · ENIAC, EDVAC, UNIVAC (assemblador, concepte de software)
1947	Transistor · 2a generació	IBM 7094, FORTRAN
1961	Circuits integrats · 3a generació	PDP-11 (primer minicomputador, 20 000 $), COBOL, LISP (alt nivell)
1974	Primer processador en xip (Intel 4004) · 4a generació	Commodore (300 $), PC
1980–2000	Memòria semiconductora, circuits VLSI, sistemes encastats	Tauletes, mòbils...

ENIAC (1943-46, Eckert i Mauchly, Pennsylvania): taules de trajectòries balístiques; 6 programadores (*Top Secret Rosies*); 167 m², 27 t, 17 468 vàlvules, 7 200 díodes, 1 500 relés, 70 000 resistències, 10 000 condensadors; 1900 sumes/s. **Programa no emmagatzemat**.

EDVAC (1945; von Neumann, Eckert, Mauchly): codi binari, **programa emmagatzemat** (arquitectura von Neumann), assemblador; 6 000 vàlvules, 12 000 díodes, 56 kW, 45,5 m², 7 850 kg. **UNIVAC** (anys 50): 1r computador de propòsit general (administració/negocis), targetes perforades, >1 M$, 48 unitats.

1a generació: màquines grans i pesades, alt consum, aturades per errors, E/S amb targetes perforades, programació en assemblador, informació en tambors magnètics.

TEMA 2 · Representació de la informació

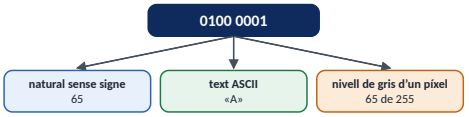
2.1 Informació · 2.2 Octal · 2.3 Bases · 2.4 Canvi de base · 2.5 Operar sense signe · 2.6 Text · 2.7 Imatge · 2.8 So · 2.9 Naturals i enters · 2.10 Reals · 2.11 Operar SM i C2 · 2.12 Carry i overflow

2.1 Representació de la informació en el computador

EN PARAULES SENZILLES

L'ordinador només «entén» dos estats elèctrics (0 o 1). Tot —números, lletres, fotos, so, programes— es guarda com una fila de 0 i 1; **codificar** és decidir quin patró de bits representa cada cosa.

Sistema digital: dos nivells (0/1 o L/H). **Text:** s'entra en format humà; la codificació no és arbitrària. **Números:** entren com a caràcters, però el seu valor posicional exigeix operar en base 2. **So:** ona captada per un micròfon → senyal analògic → digitalitzat. **Imatges:** molt voluminoses, estàtiques o dinàmiques (vídeo), se solen comprimir. **Instruccions:** s'entren com a text i es transformen en codi màquina. **El mateix patró de bits significa coses diferents: ho decideix el tipus de la dada.**



Un mateix byte: natural, lletra o gris

∑ FÓRMULA

Amb n bits: 2^n patrons diferents

Bits per a M valors: $n = \lceil \log_2 M \rceil$

PAS A PAS

Quants bits calen per representar M valors?

1. Busca la potència de 2 més petita que sigui ≥ M.
2. L'exponent és el nombre de bits n.

EXEMPLE · quants bits?

26 lletres → $2^5 = 32 \geq 26$ → **5 bits**. 100 valors → $2^7 = 128 \geq 100$ → **7 bits**. Els dígits 0-9 → $2^4 = 16$ → **4 bits** (per això BCD usa 4 bits).

2.2 Base octal

EN PARAULES SENZILLES

L'octal compta de 8 en 8 (dígits 0 a 7). Com que $8 = 2^3$, **cada dígit octal = 3 bits**: serveix per escriure fileres de bits més curtes. Ús: permisos UNIX, `chmod 755` = 111 101 101 (r-w-x amb pesos 4-2-1).

∑ FÓRMULA

$N = \Sigma\, r_i \cdot 8^i$ (r de 0 a 7)

1 dígit octal = 3 bits (pesos 4-2-1)

PAS A PAS

Octal → decimal

1. Numera els dígits de dreta a esquerra des de 0 (fracció: −1, −2...).
2. Multiplica cada dígit per $8^{\text{posició}}$.
3. Suma.

PAS A PAS

Decimal → octal

1. Part entera: divideix entre 8 fins quocient 0; residus de baix a dalt.
2. Fracció: multiplica per 8 i agafa la part entera (de dalt a baix).

PAS A PAS

Binari ↔ octal

1. Binari → octal: grups de 3 bits des de la coma (completa amb zeros), pesos 4-2-1.
2. Octal → binari: cada dígit en 3 bits.

EXEMPLE · octal

$705{,}51_8 = 7\cdot64 + 0\cdot8 + 5 + 5/8 + 1/64 =$ **453,640625**. $010\ 111\ 011_2 =$ **273_8**.

2.3 Sistemes de numeració (bases)

∑ FÓRMULA

$N = \ldots + r_2\cdot b^2 + r_1\cdot b^1 + r_0\cdot b^0 + r_{-1}\cdot b^{-1} + r_{-2}\cdot b^{-2} + \ldots = \Sigma\, r_i\cdot b^i$

Base B: alfabet de B símbols; cada xifra aporta el seu **valor** i el **pes** de la seva posició (B^n). Decimal {0...9} · binari {0,1} · octal {0...7} · hexadecimal {0...9, A...F}. Octal i hex = agrupar 3 o 4 bits.

EXEMPLE · avaluar amb la fórmula

$527{,}03_{10} = 5{\cdot}10^2 + 2{\cdot}10^1 + 7{\cdot}10^0 + 0{\cdot}10^{-1} + 3{\cdot}10^{-2}$

$101{,}01_2 = 4 + 0 + 1 + 0 + ¼ = \mathbf{5{,}25}$ · $A5{,}0C_{16} = 10{\cdot}16 + 5 + 0 + 12/256 = \mathbf{165{,}046875}$

2.4 Canvi de base

EN PARAULES SENZILLES

Decimal → binari: divideix entre 2 i anota els residus (de baix a dalt); per als decimals, multiplica per 2 i anota la part entera (de dalt a baix). Binari ↔ hex: talla de 4 en 4 bits.

Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

PAS A PAS

Qualsevol base → decimal

1. Escriu el pes de cada dígit: $B^{\text{posició}}$ (A…F valen 10…15).
2. Multiplica cada dígit pel seu pes i suma-ho tot.

PAS A PAS

Decimal → binari

1. **Part entera:** divideix entre 2 i anota el residu; repeteix amb el quocient fins a 0. Llegeix de l'últim (MSB) al primer (LSB).
2. **Fracció:** multiplica per 2, anota la part entera (0/1), queda't amb la decimal; repeteix fins a 0 o fins tenir els bits. Llegeix de dalt a baix (primer = MSB).

PAS A PAS

Binari ↔ hexadecimal (generalitzable a bases 2^k)

1. Binari → hex: grups de 4 bits des de la coma (completa amb zeros); suma pesos 8-4-2-1 de cada grup (10 = A … 15 = F).
2. Hex → binari: cada dígit en 4 bits. Decimal → hex directe: divisions entre 16.

EXEMPLE · → decimal

$11011_2 = 16+8+2+1 = \mathbf{27}$

$101{,}11_2 = 4+1+½+¼ = \mathbf{5{,}75}$

$13E_{16} = 256+48+14 = \mathbf{318}$

$20F{,}C_{16} = 512+15+12/16 = \mathbf{527{,}75}$

EXEMPLE · 18,625 → binari

Entera: 18=9·2+**0**; 9=4·2+**1**; 4=2·2+**0**; 2=1·2+**0**; 1=0·2+**1** → **10010**

Fracció: 0,625·2=1,25→**1**; 0,25·2=0,5→**0**; 0,5·2=1,0→**1** → **,101**

Resultat: $\mathbf{10010{,}101}_2$

EXEMPLE · hex ↔ bin ↔ dec

$5B{,}A8_{16} = \mathbf{0101\ 1011{,}1010\ 1000}_2$

100 1101,1010 01_2 → 0100

1101,1010 0100 = $\mathbf{4D{,}A4}_{16}$

318 → hex: 318÷16 = 19 r **14(E)**;

19÷16 = 1 r **3**; → **13E**

2.5 Operar sense signe

PAS A PAS

Sumar en binari

1. Alinea els nombres a la dreta (mateixa mida).
2. Columna a columna: 0+0=0 · 0+1=1 · 1+1=10 (escric 0, porto 1) · 1+1+1=11 (escric 1, porto 1).
3. Un carry final vol dir que el resultat no cap en n bits. En hex igual (base 16).

PAS A PAS

Restar sense signe

1. Resta columna a columna; si el dígit de dalt és menor, **demana un manlleu** a l'esquerra (val 2 en binari, 16 en hex).
2. Només té sentit si minuend ≥ subtrahend; si no, cal representació amb signe (2.9).

EXEMPLE · suma 4 bits

0101 (5) + 0100 (4) = **1001** (9)

EXEMPLE · resta 8 bits

1100 0101 (197) − 0001 1000 (24) = **1010 1101** (173)

EXEMPLE · hex (2 bytes)

403A + E03B = **12075** (carry: no cap en 16 bits)

403A − 303B = **0FFF** (4095)

2.6 Representació de text

Caràcters: alfabètics, numèrics, especials, geomètrics/gràfics i de control (BEL, CR, LF). Codificar = assignar un codi binari a cada caràcter.

BCDIC (6 bits): 2 bits de zona + 4 numèrics. **ASCII** (7 bits, 128 codis; ANSI X3.4 / ISO 646): nombres, majúscules/minúscules, control i especials; el més usat en transmissió. **ASCII estès** (8 bits): accents i gràfics (Latin-1 = ISO 8859-1, Europa occidental). **Unicode** (ISO/IEC 10646): repertori universal; cada caràcter té un *code point* (á = U+00E1; A = U+0041) de U+0000 a U+10FFFF

(21 bits). Codificació física: **UTF-8** (1-4 bytes, compatible amb ASCII: 1 = ASCII, 2 = accents europeus, 3 = xinès/japonès/coreà, 4 = emojis) o UTF-16/32 (mida fixa, menys eficient).

∑ FÓRMULA

ASCII: 7 bits → 2^7 = 128 codis · Unicode: 21 bits · BCD numèric: 4 bits per dígit decimal

PAS A PAS

Codificar un caràcter en UTF-8

1. Troba el *code point* i passa'l a binari.
2. Tria bytes: fins 0x7F → 1 · 0x7FF → 2 · 0xFFFF → 3 · més → 4.
3. Omple la plantilla: 1: `0xxxxxxx` · 2: `110xxxxx 10xxxxxx` · 3: `1110xxxx 10xxxxxx 10xxxxxx` · 4: `11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`

PAS A PAS

Nombre decimal → BCD

1. Separa el nombre en dígits decimals.
2. Escriu cada dígit amb 4 bits (0 = 0000 … 9 = 1001) i junta els grups.

EXEMPLE · UTF-8 de «á»

U+00E1 = 1110 0001 (8 bits) → 2 bytes: 110**00011** 10**100001** = **C3 A1**

EXEMPLE · BCD

368 → 0011 0110 1000

0111 0010 1001 → **729**

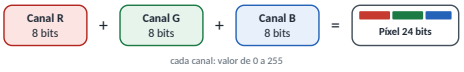
EXEMPLE · text o número?

Per teclat «12345» és text (ASCII, 1 byte/xifra): `31 32 33 34 35` = 5 bytes (3 paraules de 16 bits). Com a número: 12345 = `0x3039` = 2 bytes (1 paraula). Així no es pot operar: «0»+«1» = 48+49 ≠ 1. Ho decideix el **tipus de la dada**.

2.7 Representació d'imatge

Es captura amb càmera, vídeo o escàner i s'emmagatzema com un patró de bits. **Vectorial:** dibuixos geomètrics (línies, cercles) → poc espai (DXF d'AutoCAD, EPS). **Mapa de bits:** es divideix en **píxels** numerats per fila/columna; cada píxel guarda un atribut (B/N, escala de grisos, color; RGB 3×8 bits). **Resolució** = files × columnes. Color = percepció de la longitud d'ona (es combinen R, G, B). Més resolució o més profunditat de color → fitxer més gran; es pot guardar amb o sense compressió.

Format	Compressió	Profunditat	Ús
TIFF	opcional, sense pèrdues	1–64 bits	alta qualitat, escàners
RAW	sense pèrdues	48 bits	càmeres digitals
JPEG	amb pèrdues	8–24 bits	càmeres, Internet
GIF	sense pèrdues	1–8 bits	Internet, logos
PNG	sense pèrdues	1–48 bits	Internet, gràfics
PSD/PSP/XCF	sense pèrdues	1–64 bits	edició



Píxel RGB = 3 canals de 8 bits

∑ FÓRMULA

Mida (bits) = files × columnes × bits/píxel

RGB = 3×8 = 24 · gris = 8 · B/N = 1 bit/píxel

PAS A PAS

Mida d'una imatge sense compressió

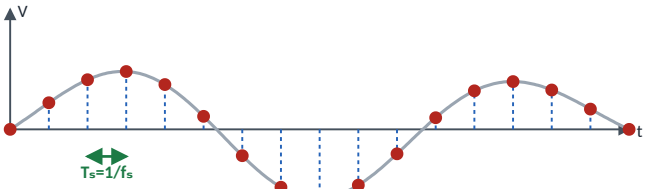
1. Píxels = files × columnes.
2. Multiplica pels bits per píxel.
3. Divideix entre 8 per tenir bytes.

EXEMPLE · imatge 1024×768

RGB: 1024·768·3 = **2 359 296 B = 2,25 MiB** · gris 8 bits: 786 432 B = 768 KiB · B/N: 98 304 B = 96 KiB.

2.8 Representació de so

El micròfon dóna un senyal analògic continu que es **mostreja** a freqüència f_s (una mostra cada $T_s = 1/f_s$); cada mostra es converteix a binari (ADC). **Qualitat** (resolució temporal): $f_s > 2 \cdot f_{màx}$ (Nyquist-Shannon). **Precisió** (resolució d'amplitud): bits/mostra.



Cada punt vermell és una mostra

Qualitat	Bits/mostra	f_s	T_s
Telèfon (PCM)	8	8 kHz	125 µs
Telèfon de qualitat	8	11,025 kHz	90,7 µs
Ràdio	8	22,05 kHz	45,4 µs
CD (per canal)	16	44,1 kHz	22,7 µs
DAT professional	24	96 kHz	10,4 µs

CODECs: PCM (un tren de polsos per mostra) → **DPCM** (guarda la diferència amb la mostra anterior: menys bits) → **ADPCM** (prediu el valor següent i guarda la diferència predit/real) → **MPEG capa III** (MP2, MP3, AAC): varia els bits segons la freqüència del senyal i la sensibilitat de l'oïda.

∑ FÓRMULA

$$f_s > 2 \cdot f_{màx} \cdot T_s = 1/f_s$$

$$\text{Mida} = \text{durada} \cdot f_s \cdot \text{bits/mostra} \cdot \text{canals}$$

PAS A PAS

Mida d'un so sense compressió

1. Comprova Nyquist: $f_s > 2 \cdot$ freqüència màxima.
2. Mostres = durada (s) × f_s.
3. Multiplica pels bytes/mostra (bits/8) i pels canals (estèreo = 2).

EXEMPLE · cançó de 2 min 30 s, estèreo, qualitat CD

150 s · 44 100 mostres/s · 2 B · 2 canals = **26 460 000 B ≈ 25,2 MiB** (26,5 MB).

2.9 Naturals i enters (SM, C1 i C2)

EN PARAULES SENZILLES

El C2 és com el comptador de quilòmetres d'un cotxe: si a 0000 li restes 1 passes a 1111, que és −1. Per això l'ordinador resta sumant i el zero és únic.

Naturals: n bits → $[0, 2^n-1]$. **Enters:** el bit més significatiu és el signe (0 positiu, 1 negatiu); si $N \geq 0$ totes les codificacions escriuen N en binari amb n bits. Classificació: naturals · enters (signe i magnitud, complement a 1, complement a 2). Altres: esbiaixada (excés; per a l'exponent dels reals) i BCD.

∑ FÓRMULA

$$\text{SM i C1: } \pm(2^{n-1}-1) \cdot \text{C2: } [-2^{n-1}, 2^{n-1}-1]$$

$$C1(N) = (2^n-1) - |N| \cdot C2(N) = 2^n - |N| \text{ (si } N < 0) \cdot C2 = C1 + 1 \cdot C2(C2(N)) = N$$

Bits	Nat.	SM	C1	C2
0000	0	+0	+0	0
0001	1	1	1	1
0010	2	2	2	2
0011	3	3	3	3
0100	4	4	4	4
0101	5	5	5	5
0110	6	6	6	6
0111	7	7	7	7
1000	8	−0	−7	−8
1001	9	−1	−6	−7
1010	10	−2	−5	−6
1011	11	−3	−4	−5
1100	12	−4	−3	−4
1101	13	−5	−2	−3
1110	14	−6	−1	−2
1111	15	−7	−0	−1

PAS A PAS

Naturals: decimal → binari de n bits

1. Passa el nombre a binari (2.4).
2. Si té menys de n bits, afegeix zeros a l'esquerra.

PAS A PAS

Signe i magnitud (negatiu)

1. |N| en binari amb n−1 bits.
2. Afegeix un 1 a l'esquerra (signe).

PAS A PAS

Complement a 1 (negatiu)

1. |N| en binari amb n bits.
2. Inverteix tots els bits.

PAS A PAS

Complement a 2 (negatiu)

1. |N| en binari amb n bits.
2. **Mètode A:** inverteix tots els bits i suma 1.
3. **Mètode B:** des de la dreta, deixa igual fins al primer 1 (inclòs) i inverteix la resta.
4. Comprovació: llegit com a natural val $2^n - |N|$.

PAS A PAS

Llegir un C2 (bits → decimal)

1. Bit de signe 0: es llegeix com un natural.
2. Bit de signe 1: fes-ne el C2 per obtenir la magnitud i posa signe menys (o calcula valor sense signe − 2^n).

Mètode	Regla si N < 0	Doble zero?	Rang (n bits)
Signe i magnitud	bit signe 1 + |N|	sí	$\pm(2^{n-1}-1)$
Complement a 1	invertir tots els bits	sí	$\pm(2^{n-1}-1)$
Complement a 2	invertir bits + sumar 1	no (estàndard)	$[-2^{n-1}, 2^{n-1}-1]$

EXEMPLE · n = 6, N = −12

+12 = 001100 · SM(−12) = **1**01100 · C1(−12) = **110011** · C2(−12) = 110011 + 1 = **110100** (comprovació: 110100 = 52 = 64 − 12 ✓)

EXEMPLE · n = 8, N = −24

|24| = 0001 1000 · C1 = 1110 0111 · C2 (A) = 1110 0111 + 1 = **1110 1000**. Mètode B: des de la dreta «1000» queda igual i la resta s'inverteix → 1110 1000 ✓. Rangs n=8: natural 0…255, SM/C1 ±127, C2 −128…127.

2.10 Nombres reals: punt fix i punt flotant

∑ FÓRMULA

$$N = \pm m \cdot B^e \cdot \text{IEEE 754: } N = (-1)^s \cdot 1{,}m \cdot 2^{e-\text{bias}}$$

$$e = E + \text{bias} \cdot \text{bias} = 2^{n-1} - 1 \text{ (float: 127 · double: 1023)}$$

Punt fix: n = p + q bits (p part entera, q fraccionària), punt implícit i fix; el bit més significatiu és el signe; el rang el donen els bits enters, la precisió els fraccionaris. **Punt flotant:** $N = \pm m \times B^e$ (mantissa, exponent, base). Per evitar l'ambigüitat es **normalitza**: 1 ≤ mantissa < 2 (xifra més significativa a les unitats); l'exponent es guarda **esbiaixat** (excés a $2^{n-1}-1$) i el signe val 0/1.



IEEE 754: precisió simple (float) i doble (double)

PAS A PAS

Passar a punt fix (p, q)

1. Part entera a binari, amb zeros fins a p bits.
2. Part fraccionària amb ×2, fins a q bits.
3. Punt implícit entre les dues parts (signe al davant).

PAS A PAS

Codificar a float (IEEE 754)

1. Passa el nombre a binari.
2. **Normalitza**: 1,xxxx × 2^E.
3. **Exponent:** e = E + 127, en 8 bits.
4. **Mantissa:** els xxxx fins a 23 bits (l'1 inicial no es guarda).
5. **Signe:** 0 positiu, 1 negatiu. Junta S · e · m.

PAS A PAS

Descodificar un float

1. Separa els 32 bits en S · e · m (1·8·23).
2. E = e − 127.
3. Reconstrueix 1,m × 2^E, mou la coma E posicions i passa a decimal (signe segons S).

Cas	Signe	Exponent	Mantissa
Zero	0 / 1	0000 0000	0…0
Denormalitzat (1 no implícit)	·	0000 0000	≠ 0
±Infinit	0 / 1	1111 1111	0…0
NaN	·	1111 1111	≠ 0
Major normalitzat	0	1111 1110	1…1
Menor normalitzat	0	0000 0001	0…0

EXEMPLE · 12,25 en punt fix (p=5, q=3)

12 = 01100 · 0,25 = ,010 → **01100,010**.
−105,321 (p=8, q=8): 105 = 01101001; 0,321 → 01010010.
SM = **11101001,01010010**; C2 = **10010110,10101110**.

EXEMPLE · 22,75 → float

22,75 = 10110,11 = 1,011011 × 2^4 → e = 4+127 = 131 = 1000 0011; mantissa 011011 + **17 zeros** (fins a 23).
0 | 10000011 | 011011 0…0 = **0x41B60000**.
Descodificar-ho: E = 131−127 = 4 → 1,011011×2^4 = 22,75 ✓.

EXEMPLE · $-60\cdot2^{-8}$ → float

60 = 111100 = 1,11100 × 2^5 → ×2^{-8} → 1,111 × 2^{-3}; e = −3+127 = 124 = 0111 1100.
1 | 01111100 | 111 0…0 = 1011 1110 0111 0… = **0xBE700000**.

EXEMPLE · exponent no de base 2: $-17\cdot10^{-15}$

$10^{-15} = 2^x$, $x = -15\cdot\log_2 10 \approx -49{,}83$ → $-17\cdot2^{-49{,}83} = -17\cdot2^{-49}\cdot2^{-0{,}83} \approx -9{,}563\cdot2^{-49} = -1001{,}1001\ldots\cdot2^{-49} = -1{,}0011001\ldots\cdot2^{-46}$. e = −46+127 = 81 → **1 | 01010001 | 00110010000001001100100**.

2.11 Operar en signe i magnitud (SM) i en C2

∑ FÓRMULA

A − B = A + C2(B) · Overflow (C2): dos operands del mateix signe donen resultat de signe contrari

PAS A PAS

Sumar/restar en SM (no és directa)

1. Separa el signe de la magnitud. **Mateix signe:** suma les magnituds i conserva el signe.
2. **Signes oposats:** resta la magnitud petita de la gran i conserva el signe de la **gran**.
3. Comprova que la magnitud cap en n−1 bits.

PAS A PAS

Sumar i restar en C2 (directa)

1. Si és una resta: A − B = A + C2(B).
2. Suma bit a bit **sense distingir el signe**.
3. **Descarta el carry final** (bit n+1).
4. Comprova l'overflow (2.12).

EXEMPLE · SM (4 bits: 1 de signe + 3 de magnitud)

5 + 1: 0101 + 0001 = **0110** (6)
5 + (−6): 0101 + 1110, signes oposats → 6 − 5 = 1, signe de 6 (−) → **1001** (−1)
5 + 4 = 9: la magnitud 1001 no cap en 3 bits → **resultat incorrecte** (0001!)
65 − 33 (8 bits) = 0100 0001 − 0010 0001 = **0010 0000** (32)

EXEMPLE · C2

5 + 1 = 0101 + 0001 = **0110** (6)
−3 + 4 = 1101 + 0100 = **10001** → descarta el carry → **0001** (1)
59 − 24 = 59 + C2(24) = 0011 1011 + 1110 1000 = **10010 0011** → **35**

2.12 Carry (carreteig) i overflow

Carry: l'1 que «portem»; pot sortir del rang d'operativitat. En **naturals**, un carry final vol dir que excedim la capacitat. **Overflow** (sobreeiximent) **en C2:** el resultat surt amb el signe canviat i no és el que tocaria (dos positius donen negatiu o dos negatius donen positiu). **Un carry NO implica overflow.**

PAS A PAS

Detectar l'overflow en una suma en C2

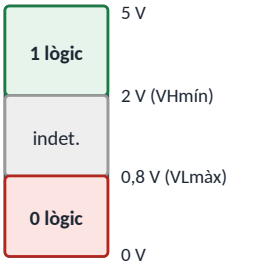
1. Mira els bits de signe dels dos operands.
2. Si són **diferents**, no pot haver-hi overflow.
3. Si són **iguals** i el del resultat és **diferent**, hi ha overflow.

Suma (8 bits)	Carry?	Resultat	Conclusió
89 + 31 (natural o C2)	no	0111 1000 = 120	correcte, sense overflow
131 + 127 (naturals)	sí	1 0000 0010	excedeix la capacitat (256+2)
−39 + (−65) (C2)	sí	1 1001 1000 = −104	carry descartat; **correcte**, sense overflow
−64 + (−64) (C2)	sí	1 1000 0000 = −128	signe igual al dels operands (1): **correcte** (−128 és representable)
−103 + (−65) (C2)	sí	1 0101 1000 = +88	**overflow**: dos negatius donen positiu (hauria de ser −168)

TEMA 3 · Components digitals

3.1 Àlgebra de Boole · 3.2 Funcions lògiques · 3.3 Portes lògiques · 3.4 Formes canòniques · 3.5 Karnaugh · 3.6 Circuits combinacionals · 3.7 Sumadors · 3.8 Mòduls MSI · 3.9 Seqüencials

Sistema digital: processa i comunica informació digital (0,1) o (L,H). En **TTL**: ≤ 0,8 V = 0 lògic ($V_{Lmàx}$), ≥ 2 V = 1 lògic ($V_{Hmín}$) i entremig és **indeterminat** (fins a 5 V). Les dades poden ser analògiques (contínues) o digitals (nivells discrets).



Nivells de tensió TTL

3.1 Àlgebra de Boole

George Boole (1860). L'àlgebra de Boole bivalent es defineix sobre B = {0, 1} amb **suma lògica** (+, OR), **producte lògic** (·, AND) i **complement** (~, NOT). (B, ·, +, ~, 0, 1) és àlgebra booleana si compleix els axiomes: **A1** commutativa (a+b = b+a; a·b = b·a) · **A2** distributiva (a·(b+c) = a·b + a·c; a+(b·c) = (a+b)·(a+c)) · **A3** element neutre (a+0 = a; a·1 = a) · **A4** complementari (a + $\bar{a}$ = 1; a·$\bar{a}$ = 0).

∑ FÓRMULA

$$\overline{a+b} = \bar{a}\cdot\bar{b} \cdot \overline{a\cdot b} = \bar{a}+\bar{b} \text{ (De Morgan, per a n variables)}$$
$$a + a\cdot b = a \cdot a\cdot(a+b) = a \cdot a + a\cdot b = a + b \cdot a\cdot(a+b) = a\cdot b$$

Propietats: P1 **dualitat** (intercanviar + ↔ · i 0 ↔ 1) · P2 unicitat del complementari · P3 complementari dels neutres ($\bar{0}$ = 1, $\bar{1}$ = 0) · P4 **idempotència** (a+a = a, a·a = a) · P5 **frontera** (a+1 = 1, a·0 = 0) · P6 **absorció** · P7 **eliminació** · P8 associativa · P9 **De Morgan**. **XOR (⊕):** commutativa, associativa, distributiva respecte l'AND, neutre 0 (a⊕0 = a), a⊕a = 0, a⊕$\bar{a}$ = 1; a⊕a⊕…⊕a val 0 si el nombre d'a és parell i a si és senar.

PAS A PAS

Simplificar una expressió

1. Busca factors comuns i agrupa amb la **distributiva** (A2).
2. Aplica el complementari (x + $\bar{x}$ = 1), el neutre (x·1 = x), l'absorció i l'eliminació.
3. Repeteix fins que no es pugui reduir més.

EXEMPLE · simplificar

f = A·B·$\bar{C}$ + A·B·C + A·$\bar{C}$
= A·C·(B+$\bar{B}$) + A·$\bar{C}$ (factor comú, A2)
= A·C·1 + A·$\bar{C}$ (complementari A4, neutre A3)
= A·(C + $\bar{C}$) = A·1 = **A**
De 3 AND + 2 NOT + 1 OR a un simple fil.

3.2 Funcions lògiques i taula de veritat

Una **funció booleana** (lògica o de commutació) $f(x_{n-1},\ldots,x_0)$ és una transformació f: $\{0,1\}^n \to \{0,1\}$. Es representa amb una **taula de veritat de 2^n files** (una per cada combinació d'entrades). Exemple de 3 variables (8 files) a la dreta; el reutilitzem a 3.4.

A	B	C	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

3.3 Portes lògiques

EN PARAULES SENZILLES

Cada porta és una «regla» amb entrades 0/1: AND (totes han de ser 1), OR (n'hi ha prou amb una), NOT (inverteix). Les que porten un cercle són la mateixa porta amb la sortida negada.



NOT $Z = \overline{A}$ · AND $Z = A \cdot B$ · NAND $Z = \overline{A \cdot B}$ · OR $Z = A + B$ · NOR $Z = \overline{A + B}$ · XOR $Z = A \oplus B$ · XNOR $Z = \overline{A \oplus B}$

Portes lògiques en simbologia ANSI/IEEE Std 91-1984 (forma distintiva, la que s'usa a l'assignatura)

A	B	AND	NAND	OR	NOR	XOR	XNOR
0	0	0	1	0	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	0	1	0	0	1

Porta	La sortida val 1 quan...
NOT	l'entrada val 0
AND	totes les entrades valen 1
NAND	alguna entrada val 0
OR	alguna entrada val 1
NOR	totes les entrades valen 0
XOR	les entrades són diferents
XNOR	les entrades són iguals

∑ FÓRMULA

$$A \oplus B = \overline{A} \cdot B + A \cdot \overline{B} \cdot \overline{A \oplus B} = \overline{A} \cdot \overline{B} + A \cdot B \cdot \text{NOT: } Z = \overline{A}$$

NAND i NOR són universals (amb només una d'elles es construeix qualsevol funció). $A \oplus B \oplus C$ val 1 si hi ha un nombre **senar** d'uns.

3.4 Formes canòniques

∑ FÓRMULA

$$f = \Sigma m(i\ldots) \text{ (on f = 1)} = \Pi M(j\ldots) \text{ (on f = 0)} \cdot 2^n \text{ files}$$

Per a tota funció booleana hi ha dues expressions canòniques («pont» entre taula de veritat i equacions). **Mínterm:** producte de les n variables, cadascuna un cop; **1** a cada variable no complementada → **suma de mínterms Σm** (representa els 1 de f). **Màxterm:** suma de les n variables; **0** a cada variable no complementada → **producte de màxterms ΠM** (representa els 0 de f).

PAS A PAS

De la taula de veritat a la forma canònica

1. **Suma de mínterms:** per cada fila amb f = 1, escriu un producte (variable **directa** si val 1, **complementada** si val 0) i suma'ls.
2. **Producte de màxterms:** per cada fila amb f = 0, escriu una suma (variable **directa** si val 0, **complementada** si val 1) i multiplica-les.

EXEMPLE · funció de la taula 3.2

f = 1 a les files 001, 010, 100, 111 → $f = m_1 + m_2 + m_4 + m_7 = \mathbf{\Sigma m(1,2,4,7)} = \overline{A} \cdot \overline{B} \cdot C + \overline{A} \cdot B \cdot \overline{C} + A \cdot \overline{B} \cdot \overline{C} + A \cdot B \cdot C$

f = 0 a les files 000, 011, 101, 110 → $\mathbf{\Pi M(0,3,5,6)} = (A+B+C) \cdot (A+\overline{B}+\overline{C}) \cdot (\overline{A}+B+\overline{C}) \cdot (\overline{A}+\overline{B}+C)$. (Aquesta f és $A \oplus B \oplus C$.)

3.5 Mapes de Karnaugh

EN PARAULES SENZILLES

Un mapa de Karnaugh és la taula de veritat en 2D amb ordre **Gray** (00, 01, 11, 10): dues cel·les veïnes difereixen en un sol bit i per això X·Y + X·$\overline{Y}$ = X. Agrupes els 1 veïns i la funció es simplifica sola.

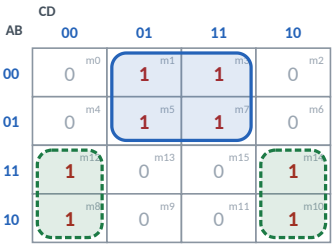
Representació bidimensional de la taula de veritat, per a simplificació visual (fins a 4 variables). **2 variables:** 2×2; **3:** 2×4 (A files, BC columnes); **4:** 4×4 (AB files, CD columnes). El mapa és **toroidal**: la primera i l'última columna (i fila) són adjacents, i les quatre cantonades són veïnes.

PAS A PAS

Simplificar amb Karnaugh

1. Dibuixa la graella amb ordre **Gray** i posa-hi els 1 dels mínterms.
2. Agrupa els 1 veïns en rectangles o quadrats de **1, 2, 4, 8 o 16** cel·les (potència de 2).
3. Fes els grups **tan grans com sigui possible** i el **mínim nombre de grups** per cobrir tots els 1; es poden solapar.
4. En cada grup, **desapareix la variable que canvia** de valor i es queden les constants (terme producte).
5. Suma els termes de tots els grups (SOP).

Don't care (X o d): combinacions d'entrada que mai es produeixen (ex. 10 a 15 en BCD) o sortides on el valor no importa. Si ajuden a fer un grup més gran, es tracten com un **1**; si no, com un **0** (s'ignoren). Mai és obligatori agrupar una X.



$f = \overline{A}\cdot D + A\cdot\overline{D} = A \oplus D$

$\overline{A}\cdot D$

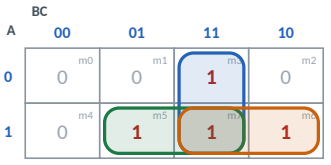
{m1,m3,m5,m7}

$A\cdot\overline{D}$

{m8,m10,m12,m14}

(toroidal: col. 00 i 10)

f = Σm(1,3,5,7,8,10,12,14) → $\overline{A}\cdot D + A\cdot\overline{D} = A \oplus D$



{m3,m7} → B·C

{m5,m7} → A·C

{m6,m7} → A·B

S = A·B + A·C + B·C

Mapa de 3 variables: S = Σm(3,5,6,7) (alarma de dipòsit); els tres grups comparteixen m7

EXEMPLE · de la teoria al mapa de 4 variables

Grup 1 (m1,m3,m5,m7): A=0 fix; B canvia (0→1); C canvia (0→1); D=1 fix → queda **$\overline{A}$·D**. Grup 2 (m8,m10,m12,m14, via cantonades): A=1 fix; B i C canvien; D=0 fix → **A·$\overline{D}$**. Resultat: $\overline{A}$·D + A·$\overline{D}$ = A⊕D (de 8 mínterms a una XOR).

Amb don't care: detector de dígit BCD senar f = Σm(1,3,5,7,9) + d(10…15): els X 11, 13, 15 completen el grup de 8 (D=1) → **f = D**.

3.6 Circuits combinacionals: anàlisi i síntesi

Un **circuit combinacional** és aquell en què les sortides depenen **únicament dels valors actuals de les entrades**. Àlgebra de Boole → teoremes → funcions booleanes → (minimització) → portes lògiques → circuits i mòduls combinacionals.

Anàlisi: obtenir les expressions entrades-sortides i determinar el comportament: **mètode algebraic** (a partir de les expressions de les portes) o **per taula de veritat** (donant valors a les entrades).

Qualsevol circuit es pot fer en **dos nivells** de portes (i inversors): **AND-OR** (suma de productes, SOP, mínterms) · **OR-AND** (producte de sumes, POS, màxterms) · **NAND-NAND** (= AND-OR) · **NOR-NOR** (= OR-AND).

PAS A PAS

Síntesi d'un circuit

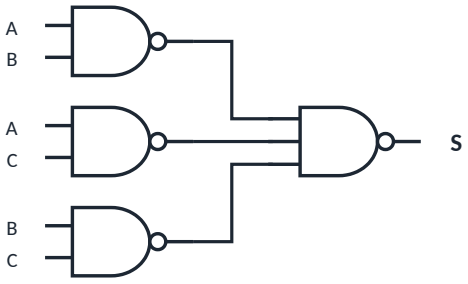
1. Determina les **entrades i sortides** a partir de les especificacions.
2. **Codifica** (a binari) la informació d'entrada i sortida, si cal.
3. Crea la **taula de veritat** que satisfà la relació funcional.
4. Obté l'expressió en **suma de productes** i **simplifica** (Boole o Karnaugh).
5. Transporta l'expressió a un circuit amb **portes lògiques** (o només NAND/NOR).

EXEMPLE · alarma industrial

Dipòsit amb 3 sensors (A pressió alta, B temperatura alta, C nivell ple). **S = 1 si com a mínim 2 sensors** detecten anomalia (funció «majoria»).

m	A	B	C	S	Sensors
0	0	0	0	0	cap
1	0	0	1	0	C
2	0	1	0	0	B
3	0	1	1	1	B i C
4	1	0	0	0	A
5	1	0	1	1	A i C
6	1	1	0	1	A i B
7	1	1	1	1	A, B i C

$S = \Sigma m(3,5,6,7) \rightarrow$ Karnaugh (fig. 3.5) $\rightarrow$ **$S = A{\cdot}B + A{\cdot}C + B{\cdot}C$**.
AND-OR: 3 portes AND de 2 entrades + 1 OR de 3.
NAND-NAND (doble negació + De Morgan): $S = \sim\{\overline{A{\cdot}B}\cdot\overline{A{\cdot}C}\cdot\overline{B{\cdot}C}\} \rightarrow$ **4 portes NAND** (3 de 2 entrades + 1 de 3).



Realització NAND-NAND

3.7 Sumadors: mig sumador i sumador complet

∑ FÓRMULA

Half adder: $S = A \oplus B$, $C_{out} = A{\cdot}B$
Full adder: $S = A \oplus B \oplus C_{in}$, $C_{out} = A{\cdot}B + B{\cdot}C_{in} + A{\cdot}C_{in}$

Suma binària: 0+0 = 0, 0+1 = 1, 1+0 = 1, 1+1 = 10 → 2 bits: carreteig (*carry*) i suma. **Mig sumador:** admet 2 bits (A, B) i dóna suma S i carry de sortida C_{out}. **Sumador complet:** admet A, B i un carry d'entrada C_{in}; **S = 1** si hi ha un nombre senar d'uns entre A, B i C_{in}; **C_{out} = 1** si n'hi ha dos o més.

A	B	S	C_{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

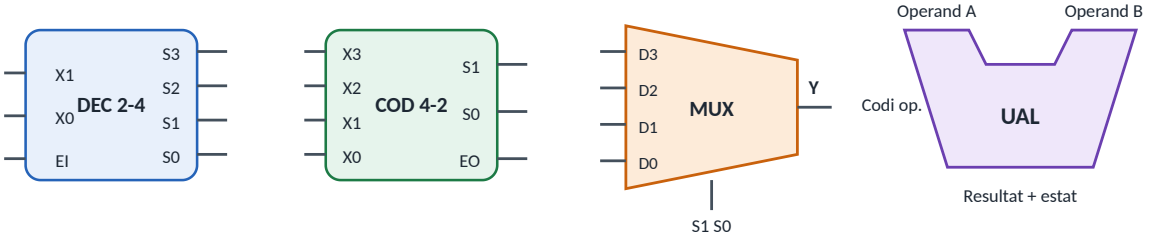
Mig sumador

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Sumador complet

3.8 Mòduls combinacionals (MSI)

Circuits integrats de mitjana escala (*Medium Scale Integration*, 10–100 portes) que fan una funció combinacional específica, predefinida i optimitzada. **Avantatges:** major nivell d'**abstracció**, **simplificació** i menys errors, **reutilització** i estandardització (sèrie 7400) i **escalabilitat** (cascada o expansió; p. ex. dos sumadors de 4 bits → 8 bits). **Tipus:** sumadors i comparadors · descodificadors, codificadors i multiplexors (encaminament i selecció de dades) · UAL/ALU.

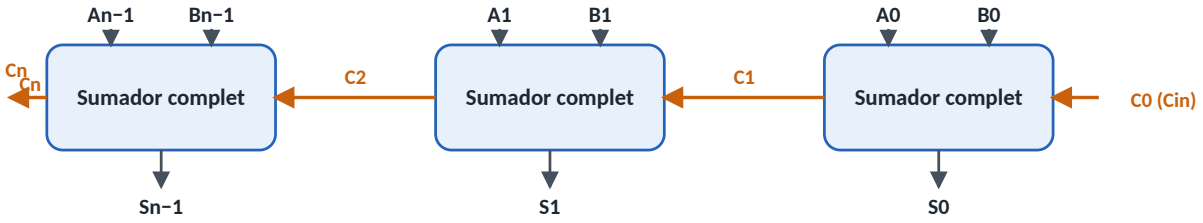


Símbols dels mòduls: descodificador 2-4, codificador 4-2, multiplexor 4-1 i UAL

3.8.1 Sumador de n bits i comparador

El **sumador de n bits** s'obté per **repetició de n sumadors complets en cascada**: el C_{out} de cada un és el C_{in} del següent (*ripple carry*). **Comparador:** compara magnituds i dóna A<B, A=B o A>B (per a 1 bit): **A<B = A·$\overline{B}$ · A=B = A⊕B · A>B = A·$\overline{B}$**.

EXEMPLE · 5 + 3 amb 4 sumadors
bit0: 1+1 = **0**, c=1 · bit1: 0+1+1 = **0**, c=1 · bit2: 1+0+1 = **0**, c=1 · bit3: 0+0+1 = **1**, c=0 → **1000** = 8.



Sumador de n bits: n sumadors complets en cascada (el carry viatja del bit 0 cap al n−1)

3.8.2 Descodificador i codificador

Descodificador (demultiplexor): n línies d'entrada i 2^n de sortida; cadascuna de les 2^n configuracions d'entrada activa **única i exclusivament** una línia de sortida. Amb **enable** EI: si EI = 0, totes les sortides a 0. **Codificador:** 2^n entrades i n sortides; les sortides presenten el **codi binari de la línia d'entrada activada** (4-a-2 amb Enable: EI i EO).

EI	X1	X0	S3	S2	S1	S0
0	X	X	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

S0 = ~{X1}·~{X0}·EI · S1 = ~{X1}·X0·EI · S2 = X1·~{X0}·EI · S3 = X1·X0·EI

EI	X3	X2	X1	X0	S1	S0	EO
0	X	X	X	X	0	0	1
1	0	0	0	1	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	1	0	0	0	1	1	0

S1 = (X3+X2)·EI · S0 = (X3+X1)·EI · EO = ~{EI}

3.8.3 Multiplexor i UAL

∑ FÓRMULA

$$DATA = A\cdot\overline{sel} + B\cdot sel \text{ (MUX 2-1)} \cdot Y = \Sigma\, D_i\cdot m_i(S) \text{ (MUX 4-1)}$$
$$Y = D_0\cdot\overline{S1}\cdot\overline{S0} + D_1\cdot\overline{S1}\cdot S0 + D_2\cdot S1\cdot\overline{S0} + D_3\cdot S1\cdot S0$$

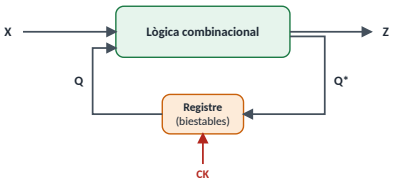
Multiplexor: selector de dades; les línies de selecció commuten una de les entrades cap a una única sortida (mux 2-1: ordre 1; mux 4-1: ordre 2). És un **mòdul universal**: qualsevol funció booleana de n variables es pot implementar amb un multiplexor de **n−1** variables de selecció (la variable restant, o una constant, va a les entrades de dades). **UAL (ALU):** circuit combinacional amb operacions aritmètiques (suma, resta, producte), lògiques (and, not… bit a bit) i desplaçament de bits (ex. xip 74LS181); entrades: operands A i B i codi d'operació; sortides: resultat i estat.

EXEMPLE · implementar z = Σm(1,2,4,5) amb un MUX 4-1
n = 3 variables → MUX amb 2 selectors (x2,x1) i la resta (x0) a les dades. Per cada parella (x2x1): 00 → m0=0,m1=1 → z = **x0**; 01 → m2=1,m3=0 → z = **x0**; 10 → m4=1,m5=1 → z = **1**; 11 → m6=0,m7=0 → z = **0**.
Connexions: S1=x2, S0=x1; **D0 = x0, D1 = x0, D2 = 1, D3 = 0**.

3.9 Circuits seqüencials

EN PARAULES SENZILLES
Un flip-flop és una memòria d'1 bit: només «mira» l'entrada D en el **flanc de pujada** del rellotge i la guarda a Q fins al flanc següent. Entre flancs, Q no canvia encara que D canviï.

Un **circuit seqüencial** té sortides que depenen de les entrades i de l'**evolució del circuit** (estat). **Síncron:** X = entrades, Z = sortides, Q = estat actual, Q* = estat següent; $Z_i = G_i(X, Q)$, $Q_i^* = H_i(X, Q)$; governat per un senyal de **rellotge** CK (senyal pulsant periòdic que activa la transició).



Circuit seqüencial síncron

∑ FÓRMULA

$$Z = G(X, Q) \cdot Q^* = H(X, Q) \cdot \text{Flip-flop D: } Q(t+1) = D$$

$$\text{Registre: load} = 1 \rightarrow Q(t+1) = I \text{ ; load} = 0 \rightarrow Q(t+1) = Q(t) \cdot \text{Comptador mòdul p: p estats}$$

3.9.1 Biestables i flip-flops

Biestable: circuit amb 2 estats que guarda 1 bit; 4 tipus: **SR, JK, D i T**. Quan és disparat pel flanc d'un rellotge s'anomena **flip-flop**. **Biestable SR** (portes NAND, entrades negades ns, nr): el biestable D sincronitzat es forma a partir de l'SR (D=1 → posa a 1; D=0 → posa a 0). **Flip-flop D** per flanc de pujada: Q(t+1) = D. **Inicialització** (asíncrona): **Preset** posa Q=1, **Reset** posa Q=0.

ns	nr	Q(t+1)
0	0	no permès
0	1	1 (set)
1	0	0 (reset)
1	1	manté Q(t)

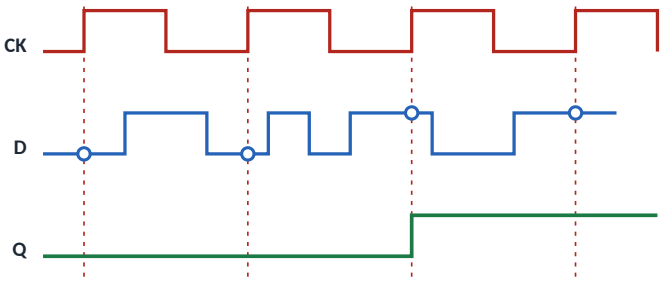


Diagrama de temps d'un flip-flop D (flanc de pujada)

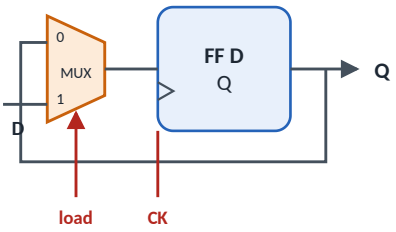
PAS A PAS

Llegir un diagrama de temps

1. Localitza els **flancs de pujada** del rellotge CK.
2. Mira el valor de **D just en aquell flanc**.
3. Q val aquest valor **fins al flanc de pujada següent**.

Al diagrama: D val 0 al 1r i 2n flanc (Q = 0), val 1 al 3r i 4t (Q = 1). Els canvis de D entre flancs s'ignoren.

3.9.2 Registre i registre de desplaçament



Flip-flop D amb multiplexor de càrrega (element repetitiu d'un registre de n bits)

Registre: circuit seqüencial síncron format per biestables que emmagatzemen un conjunt de n bits i fan transformacions funcionals. **nreset = 0** → inicialitza Q(t+1) = 0 · **load = 1** → càrrega paral·lela Q(t+1) = I · **load = 0** → manté Q(t+1) = Q(t). Un flip-flop amb un MUX al davant que controla la càrrega (load) és l'element d'un registre.

Registre de desplaçament: a més d'emmagatzemar, desplaça les dades en sèrie bit a bit. Quatre tipus segons entrada/sortida: **paral·lel/paral·lel, paral·lel/sèrie, sèrie/paral·lel, sèrie/sèrie** (entrades CK i CLR).

3.9.3 Comptadors

Comptador: circuit seqüencial la sortida del qual representa, en un codi, una seqüència ordenada que es repeteix segons els impulsos de rellotge CK. Un **comptador mòdul p** compta p estats; en arribar al darrer torna al primer. **Classificació:** per codificació (binari, decimal, Gray, anell), per forma de comptar (up, down, up/down) i per sincronisme (síncrons o asíncrons). Entrades: nreset, load, enable, CK; sortides: Q i TC. El **diagrama d'estats** representa l'evolució del circuit; el **diagrama temporal**, l'evolució dels senyals en el temps.

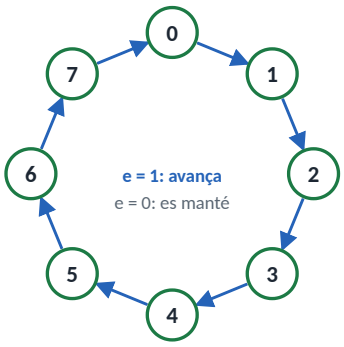


Diagrama d'estats d'un comptador mòdul 8 (e = enable)

Comptador binari ascendent mòdul 2^n: compta en binari natural (0, 1, 2, …, 2^n-1) i torna a 0; és un circuit sumador que sincronitza la propagació del carry; s'implementa amb **half-adders i flip-flops D**. **Q0** canvia a cada flanc, **Q1** cada 2, **Q2** cada 4. Un comptador mòdul p necessita **$\lceil \log_2 p \rceil$ flip-flops**.

Flanc	s0	s1	s2	s3	s4	s5	s6	s7	s0
Q2	0	0	0	0	1	1	1	1	0
Q1	0	0	1	1	0	0	1	1	0
Q0	0	1	0	1	0	1	0	1	0

EXEMPLE · quants flip-flops?
Mòdul 8 → $\lceil \log_2 8 \rceil = 3$ FF. Mòdul 10 (decimal) → $\lceil \log_2 10 \rceil = 4$ **FF**.

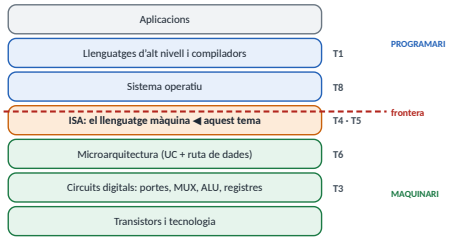
TEMA 4 · Estructura del computador i tipus d'arquitectures

4.1 On som? ISA · 4.2 Von Neumann · 4.3 Coll d'ampolla i Harvard · 4.4 La CPU · 4.5 Cicle d'instrucció · 4.6 Rendiment · 4.7 Memòria · 4.8 Busos · 4.9 ISA, CISC i RISC

Bibliografia. Bàsica: Patterson i Waterman, *Guía práctica de RISC-V* (cap. 1 = aquest tema; cap. 2 i 3 = Tema 5). Complementària: Patterson i Hennessy, *Computer Organization and Design RISC-V Edition* (cap. 1 rendiment, cap. 2 ISA) i Harris i Harris, *Digital Design and Computer Architecture: RISC-V Edition* (cap. 6).

4.1 Introducció: on som? Arquitectura i ISA

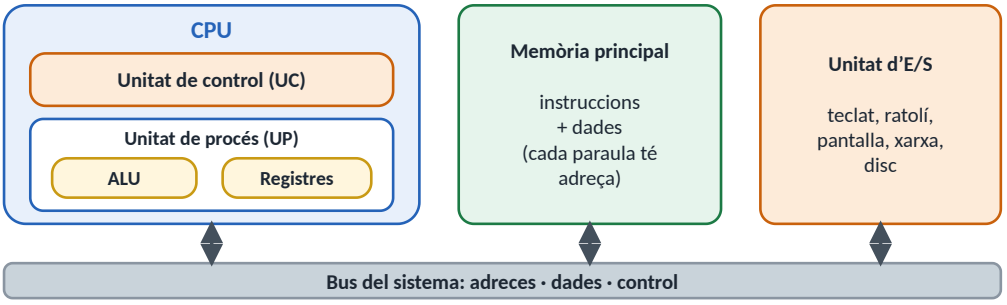
Arquitectura del computador = disseny conceptual i comportament funcional tal com el veu qui programa. El llenguatge que entén la màquina és la seva **ISA** (*Instruction Set Architecture*): **frontera entre maquinari i programari**. Una instrucció com `add` és ISA; **com la fa el circuit** (ALU, registres, UC) és **microarquitectura**. T5: programarem una ISA concreta (RISC-V); T6: com està feta per dins.



On som: l'ISA separa programari i maquinari

4.2 Model de Von Neumann: el programa emmagatzemat

1945, *First Draft of a Report on the EDVAC* (von Neumann, Eckert, Mauchly). A l'ENIAC el programa es «cablejava» (connexions i interruptors): canviar-lo costava dies. **Idea clau: el programa és una dada més.** Instruccions i dades es codifiquen en binari i es guarden a la **mateixa memòria**; canviar de programa = carregar uns altres bits. Un programa pot llegir, modificar o crear programes (compiladors, SO, carregadors). La CPU llegeix les instruccions **una darrere l'altra**; només els salts trenquen l'ordre. El que distingeix una instrucció d'una dada és **com es fa servir**.



Unitats del model: CPU (UC + UP), memòria principal, E/S i bus del sistema

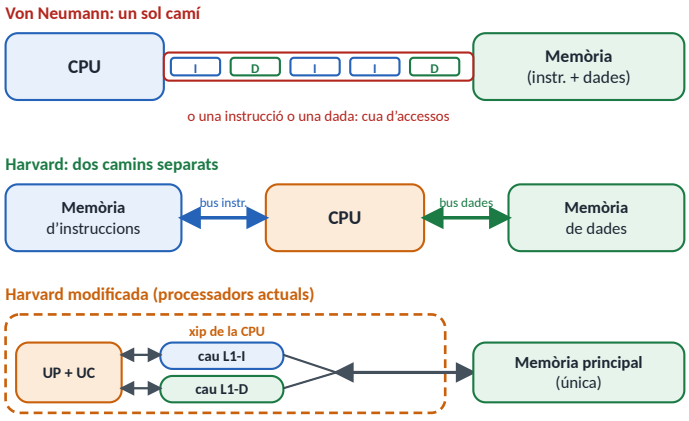
CPU: executa instruccions (la UC dirigeix i seqüencia; la UP opera amb les dades). **Memòria:** instruccions i dades en

Adreça	Contingut (exemple)
0x00	instr.: carrega A
0x04	instr.: suma B

paraules, cadascuna amb adreça. **E/S:** comunica amb l'exterior mitjançant perifèrics. **Interconnexió:** busos (4.8).

0x08	instr.: desa a C
0x0C	instr.: salta a 0x00
0x10 / 0x14 / 0x18	dades A = 25 · B = 17 · C = ?

4.3 El coll d'ampolla de Von Neumann i l'arquitectura Harvard



Un camí (Von Neumann), dos camins (Harvard) i la solució actual

Instruccions i dades viatgen pel **mateix camí**. Cada instrucció necessita un accés a memòria (portar-la) més els de les seves dades, i la CPU és molt més ràpida que la memòria → **la CPU espera**: *von Neumann bottleneck* (J. Backus, 1977). **Solucions:** dos camins separats (Harvard), **memòria cau** (T7a) i busos més amples i ràpids.

Harvard: programa i dades en **memòries i busos separats** (espais d'adreces i amplada de paraula poden diferir); llegeix instrucció i dada alhora → més velocitat, menys congestió. Inconvenient: cal dimensionar bé cada memòria. Habitual en **microcontroladors** (AVR d'Arduino, PIC) i **DSP**.

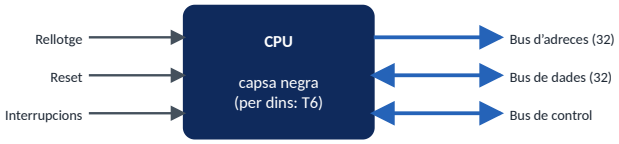
	Von Neumann	Harvard	Harvard modificada
Memòries	1 compartida	2 separades	1 principal + 2 caus L1 (I i D)
Camins (busos)	1	2	2 fins a les caus, 1 a memòria
Instr. i dada alhora	No	Sí	Sí (si són a la cau)
Programa com a dada	Sí	Difícil	Sí
Exemples	primers computadors, sistemes senzills	microcontroladors, DSP	PC, mòbils, servidors (x86, ARM, RISC-V)

Els processadors actuals són **Von Neumann** vistos pel programador (una sola memòria), però per dins tenen camins **Harvard** gràcies a les caus L1 separades (T7a).

4.4 La CPU: capsa negra, unitat de procés i registres

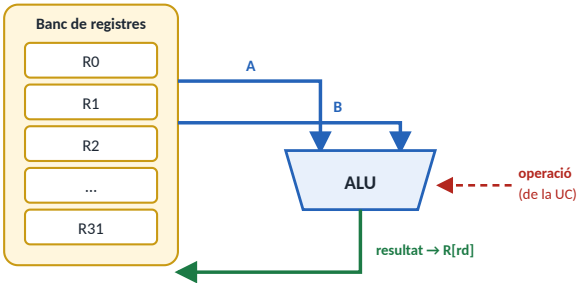
4.4.1 La CPU com a capsa negra

Llegeix instruccions de memòria, les executa i torna a començar. **Entrades:** rellotge, reset, interrupcions. **Entrades/sortides:** bus d'adreces (32), bus de dades (32, bidireccional) i bus de control.



4.4.2 ALU i banc de registres

ALU (T3): suma, resta, AND, OR, XOR, desplaçaments, comparacions; la UC decideix l'operació. **Banc de registres:** memòria molt ràpida dins la CPU (RISC-V: **32 registres de 32 bits**); en un cicle es llegeixen dos (A i B) i s'escriu un (resultat). Un registre (1 cicle) és ~**100 vegades** més ràpid que la memòria principal. Les dades van de memòria a registres, s'hi opera i, si cal, es tornen a desar.



Ruta de dades bàsica: R[rs1] → A, R[rs2] → B, ALU (operació de la UC), resultat → R[rd]

4.4.3 PC, IR i registre d'estat

∑ FÓRMULA

PC ← PC + 4 (cada instrucció ocupa 4 bytes) · Salt: PC ← adreça de destí

4.4.4 Unitat de control i rellotge

∑ FÓRMULA

$f = 1 / T_c$ · 1 GHz ↔ 1 ns · 100 MHz ↔ 10 ns

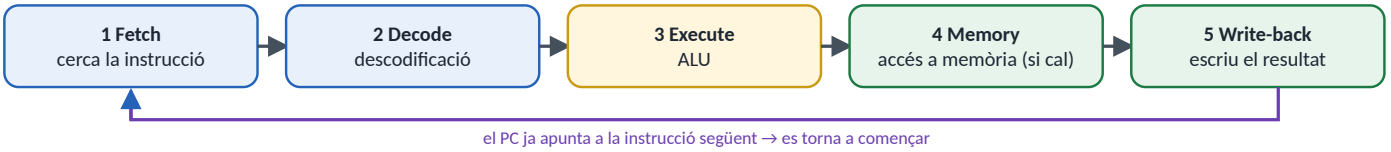
PC (*Program Counter*): adreça de la propera instrucció; els salts hi escriuen una altra adreça. **IR** (*Instruction Register*): bits de la instrucció en execució; la UC la llegeix i el programador no hi té accés. **Registre d'estat (flags):** x86 i ARM en tenen (Z zero, N negatiu, C carry, V overflow, actualitzats per l'ALU) i els salts condicionals els consulten; **RISC-V no en té**: els salts comparen directament dos registres («salta si x1 = x2»).

UC: llegeix l'*opcode* de l'IR i genera els **senyals de control** (operació de l'ALU, registres a llegir/escriure, accés a memòria, proper PC); seqüencia les fases de cada instrucció. **Rellotge:** senyal periòdic que marca el ritme; període T_c (temps de cicle) i freqüència f; els canvis es fan al flanc; una instrucció pot necessitar un o diversos cicles.

EXEMPLE · rellotge
f = 2 GHz → $T_c = 1/(2\cdot10^9)$ = **0,5 ns**.

4.5 El cicle d'instrucció

EN PARAULES SENZILLES
És com una recepta de cuina repetida mil milions de vegades per segon: **fetch** = agafar el full de la recepta (la instrucció), **decode** = entendre què demana, **execute** = fer-ho (amb l'ALU), **memory** = anar al rebost si cal, **write-back** = deixar el plat (resultat) al seu lloc (registre).



Les 5 fases (model de 3 fases: 3, 4 i 5 s'agrupen en «execute»; les 5 separades són la base de la segmentació, T6)

Fase	Què passa	RTL (X ← Y copia Y a X; M[a] = memòria; R[i] = registre)
1. Fetch	envia el PC a memòria, llegeix la instrucció i avança el PC	`IR ← M[PC] ; PC ← PC + 4`
2. Decode	la UC interpreta l'opcode i llegeix els registres font	`A ← R[rs1] ; B ← R[rs2]`
3. Execute	l'ALU opera, calcula una adreça o compara	`res ← A op B`
4. Memory (si cal)	llegeix o escriu memòria	`dada ← M[res]` o `M[res] ← B`
5. Write-back	desa el resultat en un registre	`R[rd] ← res` (o dada)

Les fases 1 i 2 són iguals per a totes; 3, 4 i 5 depenen de la instrucció. Un `add` fa fetch, decode, execute i write-back. **Salt condicional:** si es compleix la condició, PC ← destí; si no, PC ← PC + 4 (**només canvia el PC**). La CPU, des que s'engega fins que s'apaga, només repeteix aquest cicle.

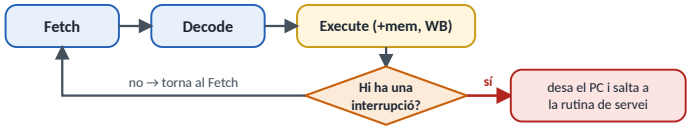
EXEMPLE · C = A + B (A=7 a 0x40, B=5 a 0x44, C a 0x48)

PC	Instrucció	Execute	Canvis
0x00	R1 ← M[0x40]	llegeix mem. 0x40	R1=7, PC=0x04
0x04	R2 ← M[0x44]	llegeix mem. 0x44	R2=5, PC=0x08
0x08	R3 ← R1+R2	ALU: 7+5	R3=12, PC=0x0C
0x0C	M[0x48] ← R3	escriu mem. 0x48	C=12, PC=0x10

Accessos a memòria: 4 (fetch) + 3 (dades) = **7**; 4 de 7 són instruccions: el coll d'ampolla de Von Neumann.

EXEMPLE · bucle: suma 3+2+1 (traça resolta)
`0x00 R1←0` · `0x04 R2←3` · `0x08 R1←R1+R2` · `0x0C R2←R2−1` · `0x10 si R2≠0: PC←0x08` · `0x14 M[0x40]←R1`
a) PC: 00, 04, 08, 0C, 10, **08, 0C, 10, 08, 0C, 10**, 14. R1: 3 → 5 → 6; R2: 2 → 1 → 0 (a la 3a volta no salta).
b) 2 + 3·3 + 1 = **12 instruccions**. **c)** 12 fetch + 1 escriptura de dada = **13 accessos**.

4.5.1 Interrupcions



La CPU comprova les interrupcions al final de cada instrucció

Un perifèric (teclat, xarxa, temporitzador…) avisa la CPU activant una **línia d'interrupció**. La CPU ho comprova **al final de cada instrucció**, mai a mitges; si n'hi ha una, **desa el PC**, executa la **rutina de servei** i continua on era. Així no cal preguntar contínuament als perifèrics.

4.6 Rendiment

EN PARAULES SENZILLES

T = quantes instruccions (N) × quants cicles triga cadascuna (CPI) × quant dura un cicle (T_c). **Amdahl:** si només una part del trajecte es pot fer en AVE, la part lenta sempre et limita el temps total.

∑ FÓRMULA

$$T = N \times CPI \times T_c = N \times CPI / f \cdot \text{Rendiment} = 1 / T \cdot A \text{ és n vegades més ràpid que B si } T(B)/T(A) = n$$

Factor	Significat	Hi influeixen...
N	instruccions **executades** (els bucles compten cada repetició)	l'algorisme, el llenguatge, el compilador i la ISA
CPI	cicles per instrucció (mitjana)	la ISA, la microarquitectura i la memòria
T_c, f	període (s) i freqüència (Hz) del rellotge	la tecnologia i la microarquitectura

PAS A PAS

Comparar dos processadors

1. Identifica N, CPI i f de cada un.
2. Calcula T = N · CPI / f.
3. Acceleració = T(lent) / T(ràpid). La freqüència sola no diu quin és més ràpid.

EXEMPLE · N = $2 \cdot 10^9$ instruccions

A (2 GHz, CPI 2): T = $2 \cdot 10^9 \cdot 2 / 2 \cdot 10^9$ = **2 s**.
B (3 GHz, CPI 4): T = $2 \cdot 10^9 \cdot 4 / 3 \cdot 10^9$ ≈ **2,67 s**.
A és 2,67/2 ≈ **1,33 vegades més ràpid** tot i tenir menys freqüència.

∑ FÓRMULA

$$CPI = \Sigma f_i \times CPI_i \quad (f_i = \text{fracció d'instruccions de la classe i})$$

$$MIPS = N / (T \times 10^6) = f / (CPI \times 10^6)$$

PAS A PAS

Calcular el CPI mitjà

1. Escriu la fracció d'instruccions de cada classe.
2. Multiplica cada fracció pel seu CPI.
3. Suma els productes.

Classe	f_i	CPI_i	$f_i \times CPI_i$
Aritmètiques i lògiques	50 %	1	0,5
Accés a memòria	30 %	2	0,6
Salts	20 %	3	0,6
Total	100 %		**CPI = 1,7**

EXEMPLE · cau millor: accessos a memòria passen a CPI = 1

Nou CPI = 0,5·1 + 0,3·1 + 0,2·3 = **1,4**. Acceleració (mateixos N i f) = 1,7/1,4 ≈ **1,21**.

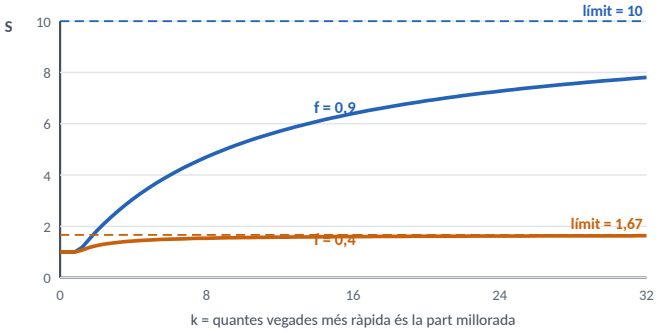
EXEMPLE · els MIPS enganyen (2 GHz)

X: N=$1 \cdot 10^9$, CPI 2 → T = **1 s**, 1000 MIPS. Y: N=$3 \cdot 10^9$, CPI 1 → T = **1,5 s**, 2000 MIPS. Y té el doble de MIPS però és **més lenta** (més instruccions per la mateixa feina). Només el temps de programes reals (*benchmarks*, SPEC CPU) és fiable: debat CISC vs RISC.

∑ FÓRMULA

$$S = 1 / ((1 - f) + f / k) \cdot S \le 1 / (1 - f) \text{ quan } k \to \infty$$

Llei d'Amdahl. S = acceleració (T abans / T després), f = fracció del temps que es pot millorar, k = quantes vegades més ràpida fem aquesta part. **Millora el cas comú:** la part no millorada (1 − f) posa el límit; per això 8 nuclis no fan un programa 8 vegades més ràpid i la memòria pesa tant.



Llei d'Amdahl: S mai supera 1/(1−f)

PAS A PAS

Aplicar la llei d'Amdahl

1. Identifica f (fracció millorable) i k (factor de millora).
2. S = 1 / ((1−f) + f/k).
3. El límit és 1/(1−f).

EXEMPLE · Amdahl

Multiplicacions = 40 % del temps, k = 10: S = 1/(0,6 + 0,04) = 1/0,64 ≈ **1,56**; límit 1/0,6 ≈ **1,67**.
Si f = 0,9, k = 10: S = 1/(0,1+0,09) ≈ **5,26**; límit 10.

4.7 La memòria vista per la CPU

EN PARAULES SENZILLES

Endianness: com escriure el nombre 1234. *Big-endian* el guarda «com el llegim» (1, 2, 3, 4 a adreces creixents); *little-endian* comença per les unitats (4, 3, 2, 1). La CPU ha de saber quin s'usa, sobretot quan enviem dades per xarxa.

∑ FÓRMULA

Capacitat = 2^n × m bits (n bits d'adreça, paraula de m bits) · adreça de la paraula i = 4·i (paraules de 32 bits)

Dada de k bytes alineada ⇔ adreça múltiple de k

Per a la CPU la memòria és un **vector**: cada posició té una adreça i guarda una paraula de m bits (32 o 64). Les **adreces són de byte**: una paraula de 32 bits ocupa 4 adreces consecutives i la seva adreça és la del primer byte (la més baixa). 32 bits d'adreça → **4 GiB**; 64 bits → **16 EiB** (2^{64} B ≈ 1,8·10^{19} B).

PAS A PAS

Alineació

1. La memòria es llegeix per **paraules senceres**.
2. Una dada de k bytes està alineada si la seva adreça és múltiple de k.
3. Una paraula no alineada ocupa dues paraules de memòria → **dos accessos** (lent o excepció).
4. El compilador alinea afegint **bytes de farciment**: es perd espai a canvi de velocitat.

Sense alinear (17 bytes) · Alineat (20 bytes)

	+0	+1	+2	+3
1000	a	b	c	h0
1004	h0	h1	h1	h2
1008	h2	w0	w0	w0
1012	w0	w1	w1	w1
1016	w1			

	+0	+1	+2	+3
1000	a	b	c	–
1004	h0	h0	h1	h1
1008	h2	h2	–	–
1012	w0	w0	w0	w0
1016	w1	w1	w1	w1

a, b, c: 1 byte · h: 2 bytes · w: 4 bytes. «–»: farciment. Sense alinear, w0 comença a 1009 → 2 accessos.

PAS A PAS

Guardar una paraula (endianness)

1. Parteix la paraula en bytes (de més a menys significatiu).
2. **Big-endian:** el byte més significatiu a l'adreça més baixa.
3. **Little-endian:** el menys significatiu a l'adreça més baixa (RISC-V, x86 i ARM).

Paraula 0x12345678 a 0x100	0x100	0x101	0x102	0x103
Big-endian (MSB a l'adreça baixa)	12	34	56	78
Little-endian (LSB a l'adreça baixa)	78	56	34	12

EXEMPLE · endianness

0x40300C01 a 0x18: big-endian 40 30 0C 01 (0x18…0x1B); little-endian **01 0C 30 40**.

Byte a 0x101 de 0x12345678: big-endian **0x34**, little-endian **0x56**.

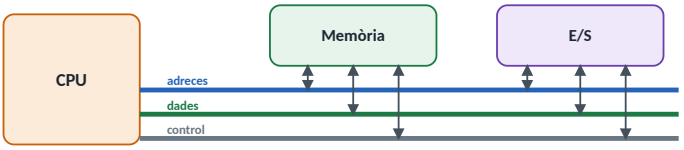
Les xarxes TCP/IP envien els enters en big-endian (*network byte order*): cal convertir-los (en C: `htonl()`, `ntohl()`).

EXEMPLE · memòria de 16 bytes

Paraula de 8 bits: 16 paraules (adreces 0x0…0xF). De 32 bits: 4 paraules a **0x00, 0x04, 0x08, 0x0C**. De 64 bits: 2 paraules a **0x00 i 0x08**.

4.8 Busos i interconnexió

Un **bus** és un conjunt de **línies** (fils) compartides que transporten bits **en paral·lel**. L'**amplada** és el nombre de línies (32 = una paraula sencera de cop). És **compartit**: només un component hi pot escriure alhora (cal **arbitratge**); tots els connectats el poden llegir. Notació: línia fina = 1 bit · línia gruixuda = bus · barra amb «n» = bus de n bits (ex. D = $\{d_{31},\ldots,d_0\}$ per un bus de 32 línies).



Els tres busos del sistema

∑ FÓRMULA

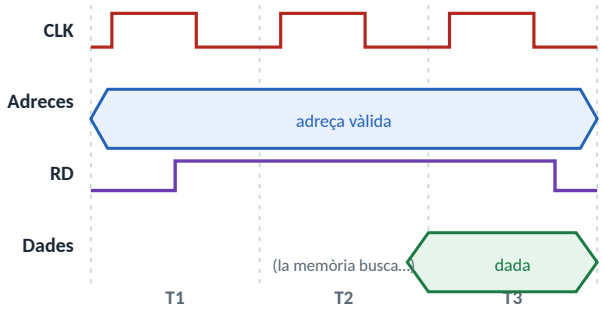
Memòria màxima adreçable = 2^n adreces (n = amplada del bus d'adreces)

Bus	Sentit	Què porta	La seva amplada determina…
Adreces	CPU → memòria, E/S	l'adreça de la posició a llegir o escriure	la memòria màxima: n bits → 2^n adreces
Dades	bidireccional	la dada llegida o escrita	quants bits per transferència
Control	bidireccional	RD, WR, rellotge, interrupcions, petició de bus…	quines operacions es poden fer

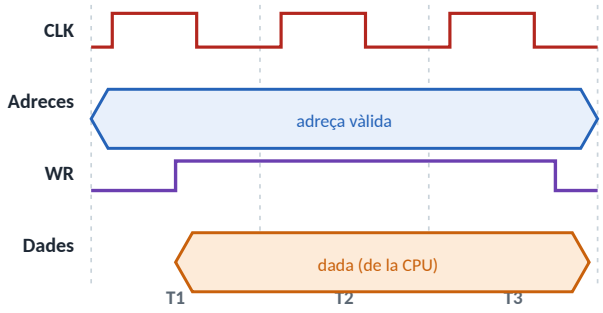
EXEMPLE · adreçament

Intel 8086: bus d'adreces de 20 bits → 2^{20} = **1 MiB**, bus de dades de 16 bits. Amb **24 bits** d'adreça: 2^{24} = **16 MiB**. Amb 32 bits: 4 GiB.

4.8.1 Una lectura i una escriptura pel bus



Lectura: la CPU llegeix de memòria



Escriptura: la CPU escriu a memòria

PAS A PAS

Lectura (RD)

1. La CPU posa l'**adreça** al bus i activa **RD**.
2. La memòria busca la dada (**tarda un temps**).
3. La memòria posa la dada al bus; la CPU la captura i desactiva RD.

PAS A PAS

Escriptura (WR)

1. La CPU posa l'**adreça i la dada** al bus i activa **WR**.
2. La memòria desa la dada a la posició indicada.
3. La CPU desactiva WR i allibera el bus.

Si la memòria és lenta, s'afegeixen **cicles d'espera** (la CPU espera més cicles T abans de capturar la dada).

4.8.2 Amplada de banda

∑ FÓRMULA

$$BW = \text{amplada (bytes)} \times f\ (Hz) \times \text{transferències per cicle}$$

Bus / enllaç	Càlcul	BW
DDR4-3200 (1 canal)	8 B × 1600 MHz × 2	25,6 GB/s
PCIe 4.0, 1 carril	16 GT/s, sèrie	≈ 2 GB/s
PCIe 4.0 ×16	16 carrils	≈ 32 GB/s

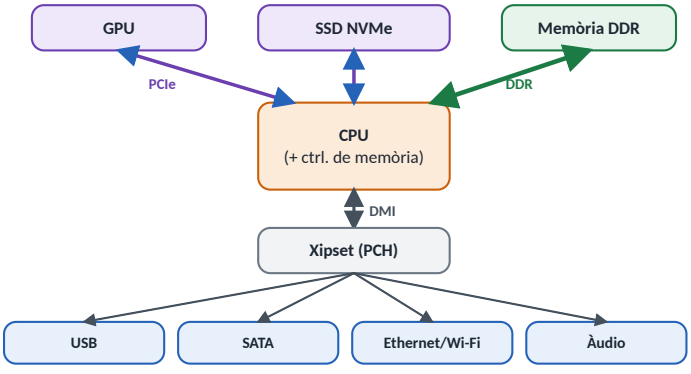
EXEMPLE · bus de 32 bits a 100 MHz, 1 transferència/cicle

a) BW = 4 B × $100 \cdot 10^6$ = **400 MB/s**.

b) 4 KiB = 4096 B → 4096 / $400 \cdot 10^6$ ≈ **10,24 µs**.

Per què els enllaços moderns són sèrie (PCIe, USB, SATA)? A freqüències altes, en un bus paral·lel els bits de cada línia arriben desfasats (*skew*) i hi ha diafonia. Un enllaç sèrie diferencial pot anar molt més de pressa.

4.8.3 Del bus únic a una jerarquia de busos



Cada tipus de trànsit té el seu enllaç

Un sol bus per a tot seria un **coll d'ampolla**: els dispositius lents frenarien els ràpids. Per això cada tipus de trànsit té el seu enllaç, a la velocitat que necessita (memòria DDR, GPU i SSD per PCIe; la resta penja del xipset).

E/S mapada a memòria: els registres dels controladors de perifèrics tenen adreces com la memòria i la CPU hi accedeix amb les instruccions normals de lectura i escriptura (**RISC-V ho fa així**). **Alternativa, E/S aïllada:** instruccions especials (x86: `IN`, `OUT`). T7b: com se sincronitzen CPU i perifèrics.

4.9 ISA: el contracte entre maquinari i programari

EN PARAULES SENZILLES

L'ISA és un **contracte**: diu què pot demanar el programari i què ha de garantir el maquinari, sense explicar com està construït el processador. És tot el que necessita saber qui programa en assemblador o qui escriu un compilador.

Registres
quants n'hi ha, de quina mida i per a què serveixen.

Tipus de dades
enters, coma flotant, mides (byte, paraula…).

Instruccions
transferència, aritmètiques i lògiques, control de flux.

Modes d'adreçament
com s'indica on és cada operand.

Format i codificació
com es converteix cada instrucció a bits.

Model de memòria
adreçament per byte, alineació, endianness (4.7).

4.9.1 Arquitectura (ISA) vs microarquitectura

Arquitectura (ISA) · QUÈ fa
- Visible per al programador: instruccions, registres, memòria.
- Si dos processadors tenen la mateixa ISA, executen els mateixos programes.

Microarquitectura · COM ho fa
- Invisible per al programador: segmentació, caus, nombre d'ALU, freqüència…
- Determina el CPI i la f (i el consum i el preu).

ISA	Microarquitectures (implementacions) diferents
x86-64	Intel Core, Intel Atom, AMD Ryzen (Zen): el mateix executable funciona a tots
ARMv8 / ARMv9	Apple M, Qualcomm Snapdragon, Cortex-A de la Raspberry Pi
RISC-V	Des de microcontroladors (ESP32-C3) fins a processadors d'aplicació (SiFive)

Analogia: l'ISA és el **volant i els pedals** d'un cotxe (tothom sap conduir-ne un); la microarquitectura és el **motor** (cada marca el fa diferent). Al T6 dissenyarem una microarquitectura senzilla per a RV32I.

4.9.2 Tipus d'ISA: on són els operands?

EN PARAULES SENZILLES
Tots els tipus fan la mateixa feina, **C = A + B** (A, B i C són a memòria), però canvia **on han d'estar les dades** abans d'operar: en una pila, en un sol registre especial, o en molts registres.

	Pila	Acumulador	Registre-memòria	Load-store

Codi	`PUSH A`
	`PUSH B`
	`ADD`
	`POP C`
		`LOAD A`
		`ADD B`
		`STORE C`
			`LOAD R1, A`
			`ADD R3, R1, B`
			`STORE R3, C`
				`LOAD R1, A`
				`LOAD R2, B`
				`ADD R3, R1, R2`
				`STORE R3, C`

Operands	al cim d'una pila	un operand és sempre el registre acumulador	un operand pot ser directament a memòria	només LOAD/STORE accedeixen a memòria; l'ALU opera amb registres
Exemples	JVM, calculadores RPN, x87	primers computadors, micros de 8 bits	x86, IBM 360, 68000	**RISC-V, ARM, MIPS**

Totes les ISA modernes de propòsit general són **de registres** (registre-memòria o load-store), perquè els registres són molt més ràpids que la memòria.

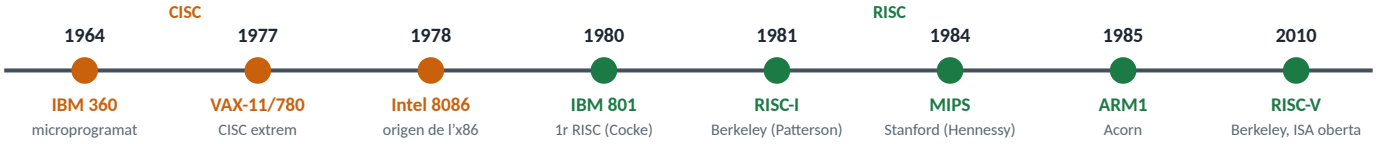
4.9.3 Modes d'adreçament

El **mode d'adreçament** indica **on és l'operand** i com se'n calcula l'adreça.

Mode	Exemple (RTL)	On és l'operand	RISC-V
Immediat	R1 ← R2 + 5	a la mateixa instrucció (constant)	Sí (`addi`)
Registre	R1 ← R2 + R3	en un registre	Sí (`add`)
Directe (absolut)	R1 ← M[0x1000]	a memòria; l'adreça és a la instrucció	No
Indirecte per registre	R1 ← M[R2]	a memòria; el registre conté l'adreça (punter)	Sí (base + 0)
Base + desplaçament	R1 ← M[R2 + 8]	a memòria, a l'adreça registre + constant	Sí (`lw`, `sw`)
Relatiu al PC	PC ← PC + 16	instrucció a una distància del PC actual	Sí (salts)
Indirecte per memòria	R1 ← M[M[R2]]	a memòria hi ha l'adreça de l'operand	No (VAX sí)

RISC-V té pocs modes; les ISA CISC (x86, VAX) en tenen molts. Al T5 veurem els de RISC-V amb la seva sintaxi. **Base + desplaçament** és el que fa servir un array: `v[2]` → M[adreça_de_v + 2·4].

4.9.4 CISC i RISC



D'on surten: dos camins, dues filosofies

CISC (*Complex Instruction Set Computer*)

- Memòria cara i lenta → **codi compacte**.
- Es programava molt en assemblador → instruccions «potents», properes a l'alt nivell.
- La microprogramació feia barat afegir instruccions.

RISC (*Reduced Instruction Set Computer*)

- Els compiladors feien servir sobretot les instruccions simples.
- Millor fer **poques instruccions simples molt ràpides** (CPI ≈ 1).
- Les caus i els compiladors millors ho fan possible. (Patterson i Hennessy: Premi Turing 2017.)

	CISC (x86)	RISC (RISC-V)
Instruccions	moltes (centenars)	poques (RV32I: ~40)
Longitud	variable (x86: 1 a 15 bytes)	fixa (32 bits)
Accés a memòria	moltes instruccions	només `load` i `store`
Modes d'adreçament	molts	pocs
Registres	pocs (x86: 8; x86-64: 16)	molts (32)
Cicles per instrucció	variable, sovint molts	≈ 1
Mida del codi	compacte (N baix)	més llarg (N alt)
Complexitat a…	el maquinari	el compilador

EXEMPLE · M[R+8] ← M[R+8] + 5

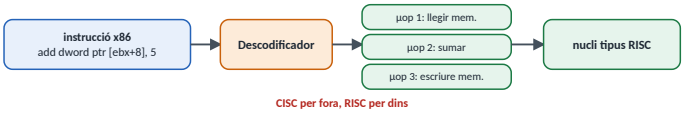
x86 (1 instrucció): `add dword ptr [ebx+8], 5`

RISC-V (3 instruccions): `lw x5, 8(x9)` · `addi x5, x5, 5` · `sw x5, 8(x9)`

El format fix de RISC-V simplifica el maquinari: els camps (registres, opcode) sempre són al mateix lloc.

4.9.5 Com són per dins i qui guanya?

CISC clàssic (anys 70-80): descodificador de longitud variable + **UC microprogramada** (microseqüenciador + ROM de microcodi) + pocs registres; gran part del xip va a la UC.
RISC: descodificador simple (32 bits fixos), UC petita (el codi d'operació es tradueix directament a senyals, T6), banc de 32 registres; l'espai es dedica a registres i execució.



x86 actual: la instrucció es trenca en micro-ops

Micro-op (µop): operació elemental de tipus RISC en què el descodificador trenca cada instrucció x86. La descodificació és cablejada per a gairebé totes; la ROM de microcodi només queda per a les més complexes.

∑ FÓRMULA

$$T = N \times CPI \times T_c$$

CISC: N ↓ (menys instruccions) però CPI ↑ · RISC: N ↑ però CPI ≈ 1 i T_c més petit

Cap guanya per si sol: depèn del producte dels tres factors. Avui la frontera s'ha difuminat (x86 és RISC per dins). **ARM** domina mòbils i sistemes encastats; **RISC-V** creix en encastats, IoT i acceleradors.
Per què RISC-V a l'assignatura? ISA **oberta i lliure** (sense llicències) · **modular**: base RV32I + extensions (M multiplicació, F/D coma flotant, C instruccions comprimides, V vectors…) · **simple i neta**, ideal per aprendre → Tema 5.